

ITA0612-Machine Learning

CO5 - AT2

Divyesh S P (192424147)

Question 1: Data Cleaning and Transformation on Time-Series Data

Aim

To perform data cleaning and transformation on time-series data by handling missing timestamps, interpolating missing values, removing outliers, and resampling the data.

Pseudocode Algorithm

BEGIN

1. Read the dataset
2. Separate input features and target variable
3. Handle missing values
4. Normalize numerical features
5. Compute correlation between features
6. Identify highly correlated feature pairs
7. For each highly correlated pair:
 - Remove one redundant feature
8. Compute correlation between each feature and target
9. Remove features with correlation below a selected threshold
10. Create a new dataset containing significant features
11. Split the data into training and testing sets
12. Train a machine learning model using selected features

13. Evaluate model performance
14. Compare dimensionality and model performance
15. Display the final selected features

END

Code

```
import pandas as pd
import numpy as np

# Load dataset
df = pd.read_csv("time_series_data.csv")

# Convert Timestamp to datetime
df["Timestamp"] = pd.to_datetime(df["Timestamp"], errors="coerce")

# Remove invalid timestamps
df = df.dropna(subset=["Timestamp"])

# Sort chronologically
df = df.sort_values("Timestamp")

# Set timestamp as index
df = df.set_index("Timestamp")

# Create complete hourly timestamp
full_range = pd.date_range(
    start=df.index.min(),
    end=df.index.max(),
    freq="h"
)

# Add missing timestamps
df = df.reindex(full_range)

# Interpolate missing values
df["Value"] = df["Value"].interpolate()

# Calculate mean and standard deviation
mean = df["Value"].mean()
std = df["Value"].std()

# Remove outliers using Z-score
z_score = (df["Value"] - mean) / std
```

```
df = df[np.abs(z_score) < 3]

# Resample data at 2-hour intervals
df_resampled = df.resample("2h").mean()

# Ensure chronological consistency
df_resampled = df_resampled.sort_index()

print("CLEANED AND TRANSFORMED DATA:")
print(df_resampled)

print("\nCHRONOLOGICAL ORDER CHECK:")
print(df_resampled.index.is_monotonic_increasing)
```

Cleaned and Transformed Data

```
CLEANED AND TRANSFORMED DATA:
                                Value
2026-01-01 08:00:00      10.00
2026-01-01 10:00:00      15.75
2026-01-01 12:00:00     109.00
2026-01-01 14:00:00      21.25
2026-01-01 16:00:00      27.50
```

Chronological Order Check

```
CHRONOLOGICAL ORDER CHECK:
True
```

Result

The time-series dataset was successfully cleaned and transformed. Missing timestamps were handled, missing values were interpolated, outliers were removed, and the data was resampled into fixed time intervals.

Question 2: Feature Selection on a Dataset

Aim

To perform feature selection by normalizing data, computing feature correlations, removing redundant features, and evaluating the selected features using a machine learning model.

Pseudocode Algorithm

BEGIN

1. Read the dataset
2. Separate input features and target variable
3. Handle missing values
4. Normalize numerical features
5. Compute correlation between features
6. Identify highly correlated feature pairs
7. For each highly correlated pair:
 - Remove one redundant feature
8. Compute correlation between each feature and target
9. Remove features with correlation below a selected threshold
10. Create a new dataset containing significant features
11. Split the data into training and testing sets
12. Train a machine learning model using selected features
13. Evaluate model performance
14. Compare dimensionality and model performance
15. Display the final selected features

END

Code

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# Load dataset
df = pd.read_csv("student_data.csv")

# Separate features and target
X = df.drop("Final_Marks", axis=1)
y = df["Final_Marks"]

# Handle missing values
X = X.fillna(X.mean())

# Normalize features
scaler = StandardScaler()
X_normalized = pd.DataFrame(
    scaler.fit_transform(X),
    columns=X.columns
)

# Compute feature correlation
correlation_matrix = X_normalized.corr()

print("FEATURE CORRELATION MATRIX:")
print(correlation_matrix)

# Remove highly correlated features
threshold = 0.90
columns_to_remove = set()

for i in range(len(correlation_matrix.columns)):
    for j in range(i):
        if abs(correlation_matrix.iloc[i, j]) > threshold:
            columns_to_remove.add(correlation_matrix.columns[i])

X_reduced = X_normalized.drop(
    columns=columns_to_remove
)

# Select features correlated with target
target_correlation = df.corr(numeric_only=True) ["Final_Marks"].abs()

significant_features = target_correlation[
```

```

        target_correlation > 0.30
].index.drop("Final_Marks")

# Keep only selected features
selected_features = [
    feature for feature in X_reduced.columns
    if feature in significant_features
]

X_final = X_reduced[selected_features]

print("\nSELECTED FEATURES:")
print(selected_features)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X_final,
    y,
    test_size=0.2,
    random_state=42
)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluate performance
score = r2_score(y_test, y_pred)

print("\nMODEL R2 SCORE:")
print(score)

print("\nNUMBER OF ORIGINAL FEATURES:")
print(X.shape[1])

print("\nNUMBER OF SELECTED FEATURES:")
print(X_final.shape[1])

```

Feature Correlation Matrix

FEATURE CORRELATION MATRIX:

	Hours_Studied	Attendance	Previous_Marks	Assignments
Hours_Studied	1.000000	0.991116	1.000000	1.000000
Attendance	0.991116	1.000000	0.991116	0.991116
Previous_Marks	1.000000	0.991116	1.000000	1.000000
Assignments	1.000000	0.991116	1.000000	1.000000

Selected Features

SELECTED FEATURES:

['Hours_Studied']

Model Performance

MODEL R2 SCORE:

0.04326530612244994

Feature Reduction

NUMBER OF ORIGINAL FEATURES:

4

NUMBER OF SELECTED FEATURES:

1

Result

The dataset was successfully normalized and analysed using correlation-based feature selection. Redundant and less significant features were removed, reducing the dimensionality of the dataset. The selected features were then used to train and evaluate a machine learning model.